

Práctica UT8: Persistencia Políglota en «GloboTour»

Víctor de Juan

Curso 2025-2026

1 Introducción: ¿Por qué NoSQL?

Tradicionalmente, hemos utilizado bases de datos relacionales (SQL) para todo. Sin embargo, en aplicaciones modernas como **GloboTour**, nos enfrentamos a desafíos que el modelo relacional no siempre puede resolver de forma eficiente:

- Datos sin esquema fijo (como descripciones de hoteles muy variadas).
- Necesidad de respuesta en microsegundos (sesiones de usuario).
- Volumen masivo de escrituras (logs de búsqueda).
- Relaciones complejas y profundas (redes sociales y recomendaciones).

En esta práctica implementaremos una arquitectura de **Persistencia Políglota**, donde utilizaremos el SGBD más adecuado para cada tarea específica.

2 Guía Docente: El Ecosistema NoSQL de «GloboTour»

Esta guía describe el flujo narrativo para explicar la UT8 utilizando el proyecto «GloboTour» como caso de estudio real.

2.1 El Problema Inicial

GloboTour empezó con MySQL, pero al crecer a nivel mundial, la base de datos relacional se convirtió en un cuello de botella.

Vamos a resolver 4 problemas críticos usando 4 paradigmas NoSQL.

2.2 Infraestructura con Docker

Para desplegar este ecosistema sin instalar cuatro servidores distintos (uno para cada paradigma NoSQL) en tu equipo, utilizaremos **Docker**.

¿Qué es Docker?

Docker permite ejecutar aplicaciones en **contenedores** aislados. Cada base de datos correrá en su propio entorno, con sus propias dependencias, pero compartiendo el núcleo del sistema operativo, lo que lo hace ligero y portable.

Gestión de Recursos de Hardware

Monitoriza el uso de RAM. Levantar los 4 simultáneos puede superar los **16 GB de RAM**. Si tu equipo es limitado, ****no los levantes todos a la vez****: levanta uno, haz las pruebas, y haz un `docker compose stop` antes de pasar al siguiente.

2.3 Módulo 1: Velocidad Extrema (Redis)

Problema: El login y el carrito de la compra van lentos. Hacer un SELECT a MySQL cada vez que un usuario navega es ineficiente.

Objetivo de la Tarea

Implementar el motor de alto rendimiento para sesiones, métricas y rankings.

Recomendación: Ver el vídeo de fireship: Redis in 100 seconds

- **Concepto a explicar:** Bases de datos en memoria y latencia.
- **Demostración:**
 1. Levantar Docker de Redis.
 2. Mostrar cómo un token de sesión `SESSION_ID` desaparece solo (TTL).
 3. Mostrar cómo crear un Ranking de las experiencias más vendidas en tiempo real.

```
cd redis && docker compose up -d
```

2.4 Módulo 2: Flexibilidad de Datos (MongoDB)

Objetivo de la Tarea

Crear un catálogo de experiencias donde cada documento tenga sus propios campos (alérgenos para comida, altura para saltos, etc.).

Problema: Tenemos experiencias muy distintas (Vuelos, Cenas, Cursos). En SQL tenemos una tabla con 50 columnas y muchos NULLs.

- **Concepto a explicar:** Esquema flexible (Schema-less) y Polimorfismo.
- **Demostración:**

1. Levantar Docker de MongoDB.
2. Mostrar el JSON de una «Cata de Vinos» vs un «Buceo».
3. Explicar que no hay que «pedir permiso» al DBA para añadir un campo nuevo.

Recomendación: Ver el vídeo de fireship: Mongodb in 100 seconds

```
# Para levantar cada servicio
cd mongodb && docker compose up -d
```

2.5 Módulo 3: Escritura Masiva y Logs (Cassandra)

Problema: Queremos guardar cada búsqueda de cada usuario para Marketing. MySQL se bloquea con tantas inserciones.

Objetivo de la Tarea

Almacenar millones de búsquedas diarias optimizando la lectura por usuario.

- **Concepto a explicar:** Escalabilidad horizontal y Query-First Design.
- **Demostración:**
 1. Levantar Docker de Cassandra.
 2. Explicar por qué NO hay JOINS: la velocidad de lectura es prioritaria.
 3. Mostrar cómo los datos se ordenan físicamente en el disco para ser leídos rápido.

Recomendación: Ver el vídeo de fireship: Cassandra in 100 seconds

```
# Para levantar cada servicio
cd cassandra && docker compose up -d
```

2.6 Módulo 4: Recomendaciones Sociales (Neo4j)

Problema: Queremos recomendar guías basados en «amigos de mis amigos». En SQL esto requiere 5-6 JOINS recursivos que matan el servidor.

Objetivo de la Tarea

Recomendar guías turísticos basándose en los gustos de los amigos del usuario.

- **Concepto a explicar:** Relaciones de primer nivel y grafos.
- **Demostración:**
 1. Visualizar la red de contactos.
 2. Ver cómo el motor «salta» de nodo en nodo instantáneamente.

Recomendación: Ver el vídeo de fireship: Neo4j in 100 seconds

```
# Para levantar cada servicio
cd neo4j && docker compose up -d
```